Team Name : Velox

Team Leader Name : Mukul

Problem Statement : The Data & AI Challenge — Intelligent Candidate Discovery & Ranking

Solution Overview

What We Built

- Evidence-weighted ranker: 7 scoring components, 100K candidates, ~60s on CPU
  - Skill 38% | Career 22% | Availability 15% | YoE 12% | Location 7% | Notice 4% | GitHub 2%
- Zero external dependencies -- pure Python stdlib, no LLMs, no APIs, no GPU
- Honeypot detection: 4 impossibility rules eliminate fabricated profiles before scoring

Why It Beats Keyword Matching

- Keywords are gamed. We weight evidence: proficiency x endorsements x duration
  - Expert in FAISS with 0 endorsements + 0 months scores LOWER than Intermediate + 40 endorsements + 36 months
- Career context: product company history, consulting-only penalty, ML signals in job descriptions
- Behavioral availability: inactive 6+ months candidates down-weighted regardless of skill match

JD Understanding & Candidate Evaluation

Key JD Requirements

- Role: Senior AI Engineer, 5-9 yrs (sweet spot 6-8), Pune/Noida/Delhi NCR/Hyderabad
- Must-have: Production embeddings (sentence-transformers, BGE, E5) + vector DB (Pinecone, Weaviate, Qdrant, Milvus, FAISS)
- Must-have: Ranking eval (NDCG, MRR, MAP) + A/B testing + strong Python
- Disqualifiers: Consulting-only career, CV/speech-only domain, no production code 18+ months

Evaluation Beyond Keywords

- Skill evidence = proficiency x log(endorsements+1)/log(51) x duration/48
- Career: product company months + ML signals in job description text + title relevance
- Tier-5 detection: text bonus for ML concepts in summary/headline even without skill tags
- Availability: last_active recency x open_to_work x response_rate x interview_completion

Ranking Methodology

Scoring Formula

- final_score = 0.38*skill + 0.22*career + 0.15*availability + 0.12*yoe + 0.07*location + 0.04*notice + 0.02*github

Skill Match (38%) -- Core Signal

- endorse_trust = log(1+n)/log(51) | duration_trust = months/48
- skill_score = weight x proficiency x (0.4 + 0.6 x evidence)
  - 60+ JD keywords, 3x weight for embeddings/vector DB/ranking/eval | Zero-evidence capped at 5% trust

Career Quality (22%)

- Product company bonus | Blacklist: TCS, Infosys, Wipro, Accenture, Cognizant, Capgemini
- ML signals in job description text | Title trajectory vs JD expectations

Availability (15%)

- Inactive >6 months max 0.35 multiplier | Low response rate penalized | open_to_work bonus

Explainability & Data Validation

How Rankings Are Explained

- Every top-100 candidate has a 1-sentence reasoning field in submission.csv
  - Format: Title | Company | YoE | Location | Skills | Response rate | Last active | Concerns
  - Example: Staff ML Engineer at Swiggy | 7.0 yrs | Delhi NCR | embeddings, vector search | 91% response rate, active 26d ago

Zero Hallucination Risk

- No LLMs in the pipeline -- all scores deterministic and field-traceable
- Reasoning assembled from factual JSON field values only, never generated

Honeypot Handling

- Pattern 1: YoE >3x implied career history | Pattern 2: 4+ expert skills, zero evidence
- Pattern 3: 10+ yrs claimed, 1 job, started <6 months ago
- Pattern 4: 3+ skills with 50+ endorsements but <=2 months duration
  - All flagged: score = 0.001 | 20 honeypots detected in 100K pool

End-to-End Workflow

Complete Pipeline: JD to Ranked CSV

- Step 1 -- Load | Read candidates.jsonl (100K records, 465MB) via gzip + json
- Step 2 -- Filter | 4 honeypot checks, flag impossibles at score = 0.001
- Step 3 -- Score | 7 components per candidate, weighted sum to final_score 0-1
  - Skill: keyword lookup + evidence weighting | Career: company + industry classification
  - Availability: recency decay + response rate | YoE: bell-curve 6-8 yr sweet spot
- Step 4 -- Rank | Sort 100K descending, tie-break candidate_id ascending
- Step 5 -- Explain | 1-sentence reasoning for top 100 from factual fields
- Step 6 -- Validate | python validate_submission.py -- Submission is valid
  - Runtime: ~60s on CPU | Peak RAM: ~2GB | No GPU | No network

System Architecture

Input Layer

- candidates.jsonl (100K records, 465MB) + hardcoded JD requirements

Filtering Layer

- Honeypot detection (4 rules) -- impossible profiles forced to score = 0.001

Scoring Layer

- score_skills() + score_career() + score_availability() + score_yoe() + score_location() + score_notice() + score_github()
- Weighted sum: final_score = sum(weight_i x component_i) for 7 components

Output Layer

- Sort top 100 -- generate_reasoning() per candidate -- write submission.csv
- validate_submission.py: format + score monotonicity + tie-break checks

Sandbox

- Gradio on HuggingFace Spaces: mukul001-redrob-ranker.hf.space

Results & Performance

Top 5 Ranked Candidates

- #1 CAND_0077337 -- Staff ML Engineer, Swiggy, Delhi NCR, 7.0 yrs | Score: 0.9539
  - Embeddings + vector search + ranking | 91% response rate | Active <30d | Notice <30d
- #2 CAND_0081846 -- Lead AI Engineer, 6.7 yrs | Score: 0.9363
- #3 CAND_0018499 -- Senior ML Engineer, 7.2 yrs | Score: 0.9327
- #4 CAND_0002025 -- Senior AI Engineer, 5.9 yrs | Score: 0.9322
- #5 CAND_0064326 -- Search Engineer, 7.6 yrs | Score: 0.9265

Compute Performance

- Runtime: ~60s | Platform: Windows 11, 8-core CPU, 16GB RAM | No GPU | No network
- Validation: python validate_submission.py -- Submission is valid
- Honeypots: 20 flagged and suppressed | Honeypot rate in top 100: 0%

Technologies Used

Core Stack -- Python stdlib only

- json + gzip -- native JSONL and .gz parsing for 465MB candidate pool
- math.log1p -- logarithmic endorsement trust scaling (no numpy needed)
- csv + datetime + pathlib + argparse -- output, recency scoring, CLI

Sandbox and Deployment

- Gradio -- live demo on HuggingFace Spaces: mukul001-redrob-ranker.hf.space
- GitHub -- github.com/Mukul07777/redrob-ranker (public, documented)

AI Tools (Declared)

- Claude (Anthropic) -- architecture discussion and code review only
  - No candidate data fed to any LLM. All scoring logic designed and validated by participant.

Why No ML Models?

- CPU-only + 5-min constraint makes embedding inference on 100K infeasible
- Rule-based: fully explainable, deterministic, zero-hallucination risk

GitHub Repository

- github.com/Mukul07777/redrob-ranker -- public, clean, documented
  - Files: rank.py | app.py | README.md | requirements.txt | submission_metadata.yaml | submission.csv | deck PDF

Live Sandbox

- mukul001-redrob-ranker.hf.space -- Gradio demo on HuggingFace Spaces
  - Paste candidate JSONL, click Rank Candidates, get ranked output with scores and reasoning

Reproduce Command

- python rank.py --candidates ./candidates.jsonl --out ./submission.csv
  - Runtime: ~60 seconds | No pip installs | No GPU | No network required

Validation

- python validate_submission.py submission.csv -- Submission is valid
  - 100 candidates | Ranks 1-100 complete | Scores non-increasing | No duplicates

INDIA.RUNS

Build what next India runs on

THANK YOU